

BANK MANAGEMENT SYSTEM

A Core Java Object-Oriented Console Application

PROJECT REPORT

Submitted as part of the VITyarthi – Build Your Own Project

Submitted By	Ayush Shukla
University	VIT Bhopal University
Course	Programming in Java
Academic Year	2026
Project	Bank Management System

1. INTRODUCTION

The Bank Management System is a console-based banking application developed using Core Java. The project demonstrates the application of Object-Oriented Programming concepts and Java's collection framework to implement basic banking operations.

The application allows users to create a new bank account, deposit money, withdraw money, and view account information through a menu-driven console interface.

The project uses a HashMap to store account objects, where the account number acts as the key and the corresponding Account object acts as the value.

The project was developed as a learning-oriented application to gain practical understanding of Java classes, objects, constructors, methods, collections, conditional statements, loops, switch-case statements, and basic validation.

2. PROBLEM STATEMENT

Managing basic bank account operations requires maintaining customer information, account numbers, and account balances while ensuring that deposits and withdrawals follow valid conditions.

The objective of this project is to develop a simple console-based Bank Management System that allows users to perform basic account operations while demonstrating the use of Java Object-Oriented Programming and the HashMap data structure.

- Create bank accounts.
- Assign unique account numbers.
- Store customer information.
- Deposit money.
- Withdraw money.
- Check account details.
- Validate basic banking operations.

3. PROJECT OBJECTIVES

- Develop a functional banking application using Core Java.
- Apply Object-Oriented Programming concepts in a practical application.
- Represent bank accounts using Java objects.
- Use HashMap for storing and retrieving account information.
- Implement account creation with automatically generated account numbers.
- Implement deposit and withdrawal operations.
- Prevent invalid transactions such as negative deposits and withdrawals exceeding the available balance.
- Provide a simple menu-driven console interface.
- Understand modular programming through classes and methods.
- Provide a foundation for future improvements such as file persistence and database integration.

4. SCOPE OF THE PROJECT

4.1 Current Scope

- Creating a new bank account.
- Automatically assigning account numbers.
- Storing account details.
- Depositing money.
- Withdrawing money.
- Checking account details.
- Searching accounts using account numbers.
- Basic transaction validation.
- Console-based interaction.

4.2 Out of Scope

- Graphical user interface.
- Database connectivity.
- User authentication.
- Online banking.
- File-based data persistence.
- Transaction history.
- ATM integration.
- Multi-user network access.

5. FUNCTIONAL REQUIREMENTS

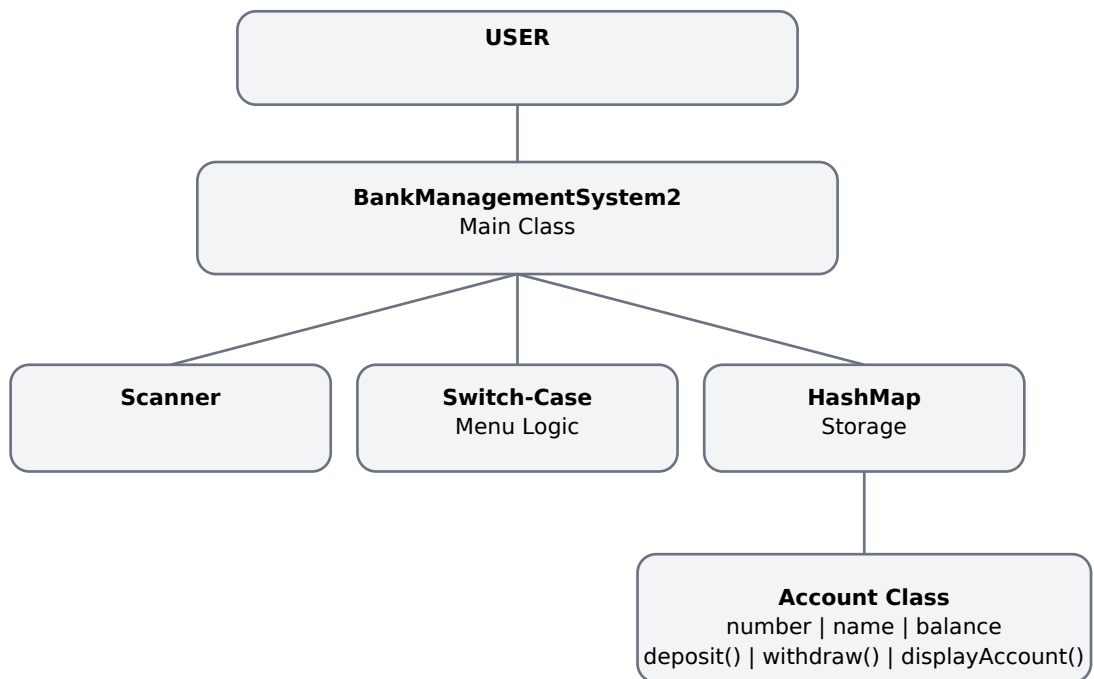
The project provides four major functional capabilities with clear input, processing, and output structures.

Module	Input	Processing	Output
Account Creation	Customer name, initial deposit	Validate deposit, generate account number, create Account object, store	Account number
Deposit	Account number, deposit amount	Check account, validate amount, update balance	Deposit confirmation and updated balance
Withdrawal	Account number, withdrawal amount	Check account, validate amount and balance, update balance	Withdrawal confirmation and remaining balance
Account Details	Account number	Retrieve Account object from HashMap	Account number, customer name, balance

6. NON-FUNCTIONAL REQUIREMENTS

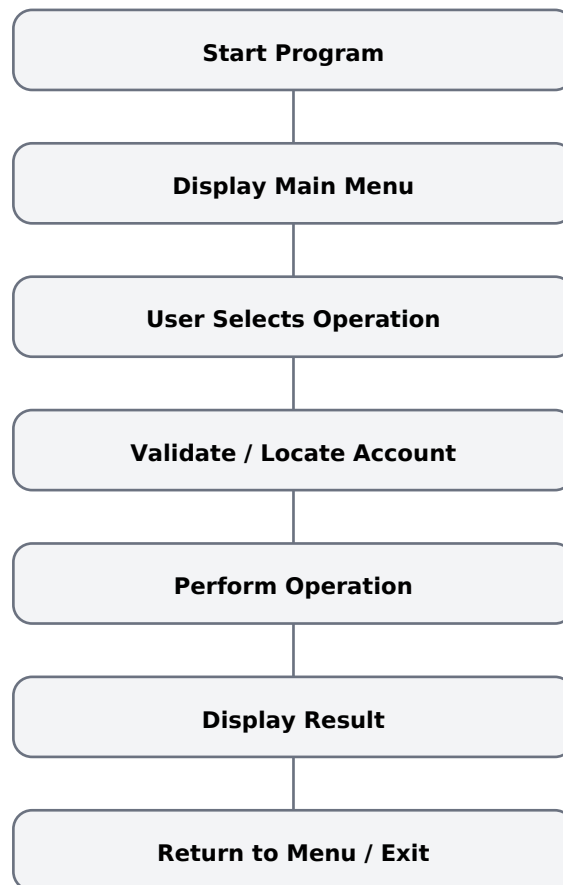
Requirement	Description
Usability	Simple menu-driven console interface for user interaction.
Performance	HashMap provides efficient account lookup using account number.
Reliability	Invalid deposits and withdrawals beyond available balance are rejected.
Maintainability	Account data and banking operations are organized in the Account class.
Error Handling	The system displays messages for invalid amounts, missing accounts, and invalid menu choices.

7. SYSTEM ARCHITECTURE



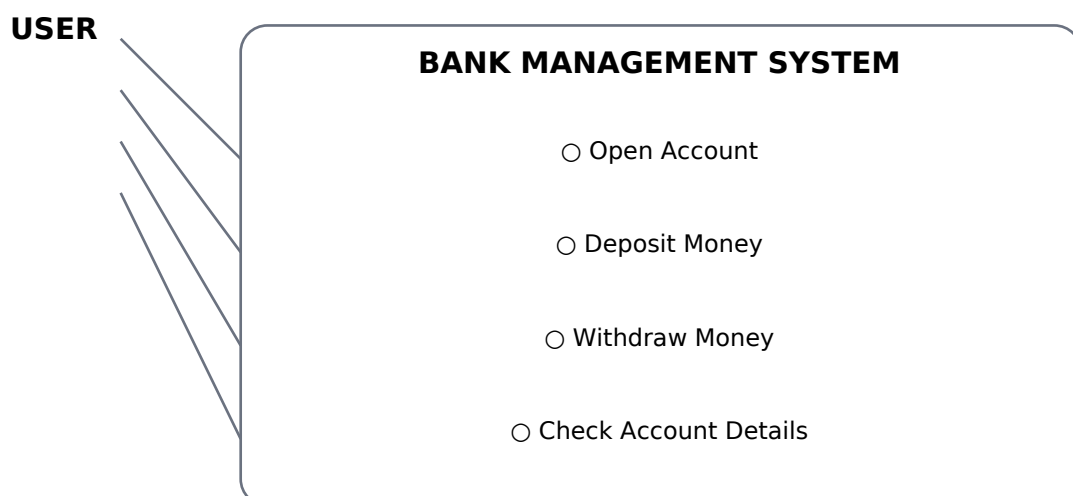
The user interacts with the main class through the console. The main class reads input using Scanner, uses switch-case to select an operation, accesses the HashMap, and invokes methods on the relevant Account object.

8. SYSTEM WORKFLOW

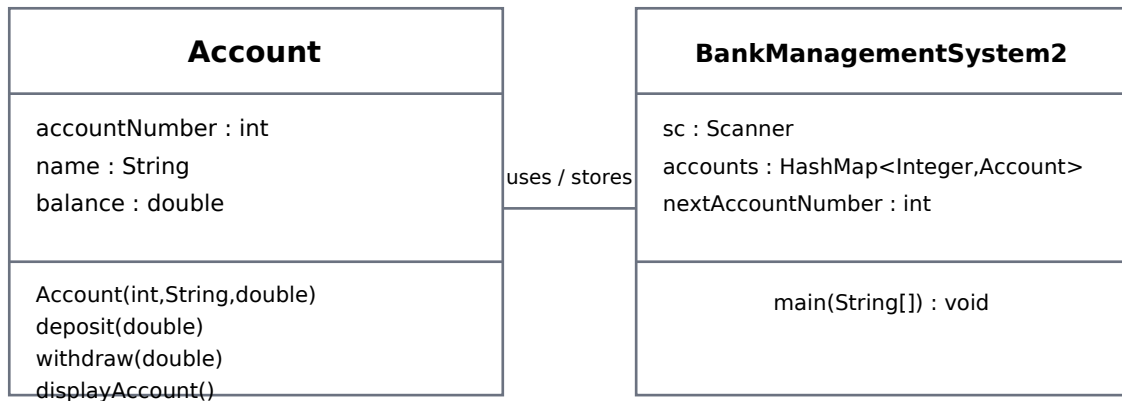


9. DESIGN DIAGRAMS

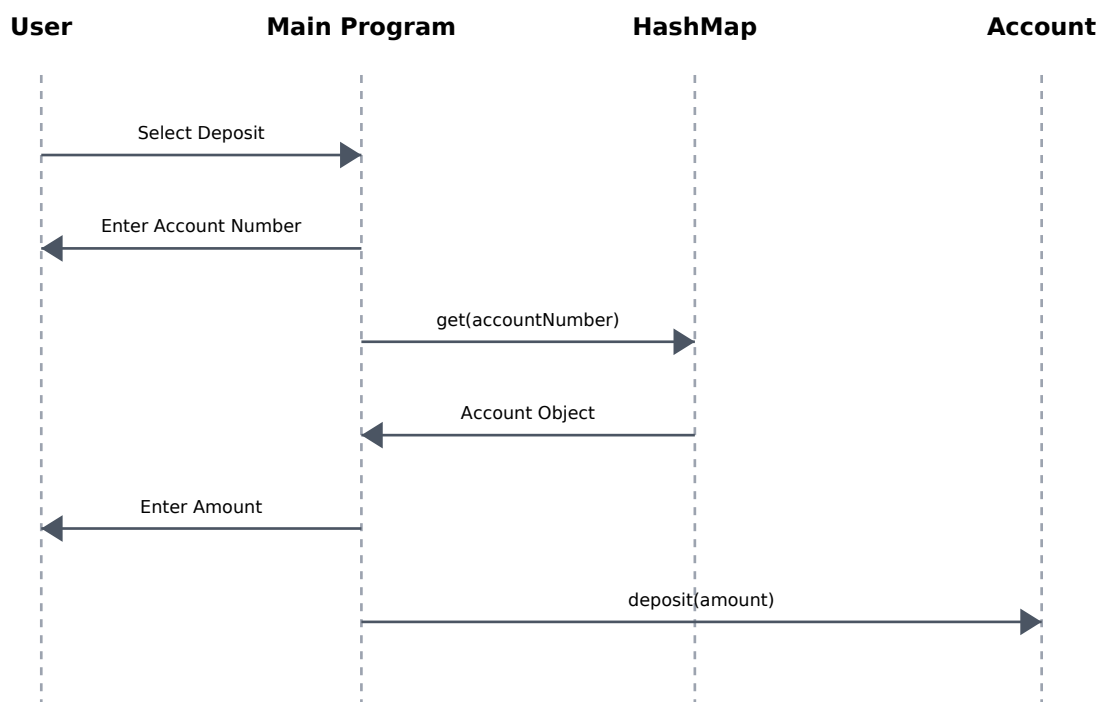
9.1 Use Case Diagram



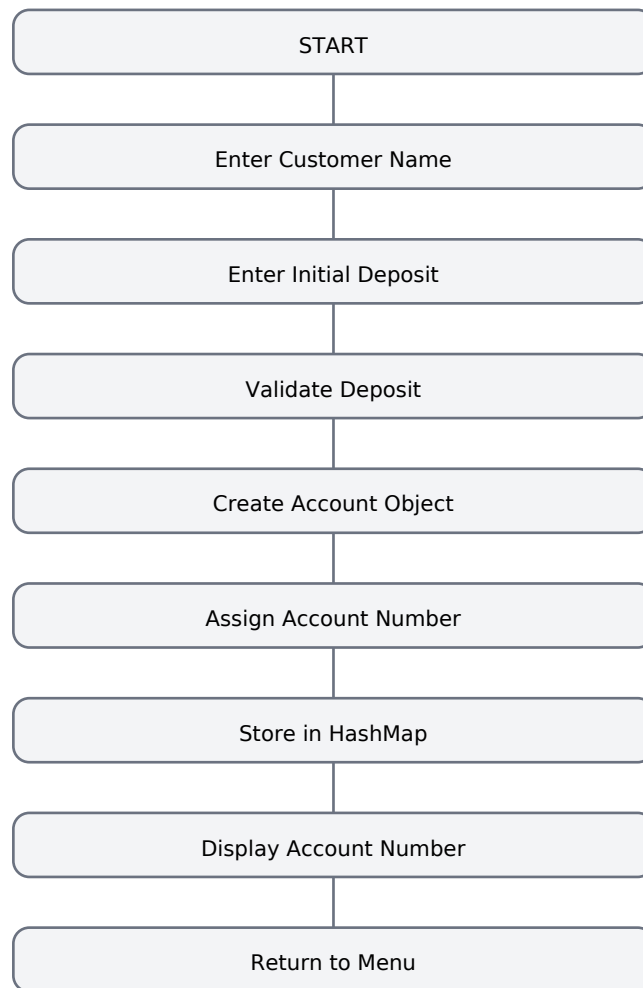
9.2 Class Diagram



9.3 Sequence Diagram — Deposit



9.4 Process Flow — Account Creation



10. DESIGN DECISIONS AND RATIONALE

Decision	Rationale
Object-Oriented design	Account-related attributes and operations are represented through a dedicated Account class.
HashMap	Account number can be used as a key to retrieve the corresponding Account object.
Scanner	Suitable for interactive console input.
Switch-case	Provides clear menu selection for banking operations.
Auto-generated account number	Starting at 1001 and incrementing provides sequential account numbers during one execution.

11. TECHNOLOGIES USED

Technology / Concept	Purpose
Java	Primary programming language.
Core Java	Application development.
Object-Oriented Programming	Class, object, constructor and method-based design.
Scanner	Keyboard input.
HashMap	Account storage and retrieval.

Technology / Concept	Purpose
Switch-case	Menu control.
Git / GitHub	Version control and project repository.

12. IMPLEMENTATION DETAILS

12.1 Account Class

The Account class contains accountNumber, name, and balance. It also contains methods for depositing money, withdrawing money, and displaying account details.

12.2 Account Constructor

The constructor Account(int accountNumber, String name, double balance) initializes an account object with its account number, customer name, and initial balance.

12.3 Deposit Operation

The deposit method checks whether the amount is greater than zero. If valid, the balance is increased using balance += amount. Otherwise, an invalid amount message is displayed.

12.4 Withdrawal Operation

The withdrawal method checks whether the amount is valid and whether it exceeds the current balance. If both conditions are satisfied, the amount is deducted using balance -= amount.

12.5 Account Storage

The main class maintains HashMap. The account number is used as the key and the Account object as the value.

12.6 Account Lookup

The application uses containsKey(accountNumber) to check whether an account exists and get(accountNumber) to retrieve the corresponding Account object.

13. DATA STRUCTURE USED

The principal data structure is Java's HashMap:

```
HashMap<Integer, Account> accounts = new HashMap<>();
```

Key	Value
1001	Account Object
1002	Account Object
1003	Account Object

14. OBJECT-ORIENTED PROGRAMMING CONCEPTS

Class

Account is a class defining the properties and behavior of a bank account.

Object

Each account created by the application is an object of the Account class.

Constructor

The Account constructor initializes account information.

Encapsulation

Account-related operations such as deposit and withdrawal are implemented inside the Account class.

Methods

deposit(), withdraw(), and displayAccount() separate individual operations into reusable units.

15. INPUT VALIDATION AND ERROR HANDLING

Condition	System Response
Negative initial deposit	Initial deposit cannot be negative.
Zero / negative deposit	Enter a valid amount.
Zero / negative withdrawal	Enter a valid amount.
Withdrawal greater than balance	Insufficient balance.
Non-existing account number	Account not found.
Invalid menu selection	Invalid choice. Please try again.

16. TESTING APPROACH

Testing was performed using normal valid inputs, invalid inputs, boundary conditions, account lookup testing, deposit testing, withdrawal testing, and menu selection testing. The test cases below represent the expected behavior of the implemented application.

17. TEST CASES AND RESULTS

ID	Scenario	Input	Expected Result
TC01	Create account	Name + ₹5000	Account created
TC02	Negative initial deposit	-₹500	Account rejected
TC03	Valid deposit	₹1000	Balance increases
TC04	Zero deposit	₹0	Invalid amount
TC05	Negative deposit	-₹100	Invalid amount
TC06	Valid withdrawal	₹1000	Balance decreases
TC07	Withdrawal greater than balance	₹10000	Insufficient balance
TC08	Negative withdrawal	-₹100	Invalid amount
TC09	Existing account enquiry	Valid account number	Account details displayed
TC10	Invalid account enquiry	Invalid account number	Account not found
TC11	Invalid menu option	9	Invalid choice message
TC12	Exit	5	Program terminates

18. EXECUTION RESULTS

Representative console outputs from the implemented functionality are shown below.

18.1 Main Menu

```
===== Welcome to Bank App =====  
1. Open Account  
2. Deposit Money  
3. Withdraw Money  
4. Check Account Details  
5. Exit
```

18.2 Account Creation

```
Enter your choice: 1  
Enter your name: Ayush  
Enter initial deposit: 5000  
Account created successfully!  
Your account number is: 1001
```

18.3 Deposit

```
Enter your choice: 2  
Enter account number: 1001  
Enter amount to deposit: 2000  
Deposit successful!  
New balance: 7000.0
```

18.4 Withdrawal

```
Enter your choice: 3  
Enter account number: 1001  
Enter amount to withdraw: 1500  
Withdrawal successful!  
Remaining balance: 5500.0
```

18.5 Account Details

```
Enter your choice: 4  
Enter account number: 1001  
Account Number: 1001  
Customer Name: Ayush  
Balance: 5500.0
```

18.6 Error Handling

```
Enter amount to withdraw: 10000  
Insufficient balance.
```

19. CHALLENGES FACED

Understanding HashMap

Understanding how keys and values work in a HashMap was an important part of the project. The account number is used as the key and the Account object as the value.

Scanner Input Handling

After `nextInt()`, the newline must be consumed before `nextLine()` is used for a customer name. The program uses `sc.nextLine()` for this purpose.

Maintaining Account Balance

Deposit and withdrawal operations must update the balance of the same Account object stored in the HashMap.

Input Validation

Banking operations must reject invalid amounts and insufficient withdrawals.

Program Flow

The do-while loop and switch-case structure allow repeated operations until the user selects Exit.

20. LEARNINGS AND KEY TAKEAWAYS

- Java classes and objects.
- Constructors and instance variables.
- Methods and object-oriented structure.
- HashMap and collection-based storage.
- Scanner-based console input.
- Conditional statements and loops.
- Switch-case menu implementation.
- Basic validation and error handling.
- Console application development.
- Git and GitHub project organization.

21. LIMITATIONS

- Account data is stored only in memory using HashMap and is lost when the program terminates.
- No database is currently integrated.
- No user authentication or PIN/password mechanism is implemented.
- The application is entirely console-based.
- Transaction history is not currently maintained.
- The current implementation does not completely handle cases where a character is entered where a numeric value is expected.

22. FUTURE ENHANCEMENTS

- File-based persistence using Java File I/O.
- Database integration using MySQL.
- User authentication and PIN protection.
- Transaction history and account statements.
- Fund transfer between accounts.
- Account types such as Savings and Current Account.
- Graphical user interface using JavaFX or Swing.

- REST API and web-based interface.
- Improved automated testing.
- Further modularization into service, validation, persistence, and transaction classes.

23. CONCLUSION

The Bank Management System demonstrates the use of Core Java and Object-Oriented Programming to implement a basic banking application. The project uses classes, objects, constructors, methods, HashMap, Scanner, loops, conditions, and switch-case control to provide account creation, deposits, withdrawals, and account enquiry.

The application provides a practical example of how programming concepts can be integrated into a functional system. Its current limitations also provide clear opportunities for future development, including persistent storage, database integration, authentication, transaction management, improved testing, and a graphical or web interface.

24. REFERENCES

- VITyarthi — Build Your Own Project: General Project Instructions & Submission Guidelines (project guideline document).
- Java Platform documentation and Core Java learning resources used during implementation.
- Udemy course followed by the author while learning and developing the basic project.

APPENDIX A — CURRENT PROGRAM MODULE SUMMARY

Component	Current Responsibility
Account class	Stores account number, customer name and balance; performs deposit, withdrawal and display operation.
BankManagementSystem2	Runs the console application, displays the menu, receives input and invokes Account operations.
Scanner	Reads user input.
HashMap<Integer, Account>	Stores account objects indexed by account number.
Switch-case	Routes menu choices to the corresponding operation.

APPENDIX B — IMPORTANT CODE EXCERPTS

Account creation:

```
Account newAccount = new Account(nextAccountNumber, name, balance);
accounts.put(nextAccountNumber, newAccount);
```

Account lookup:

```
if (accounts.containsKey(accountNumber)) {
    accounts.get(accountNumber);
}
```

Deposit:

```
if (amount > 0) {
    balance += amount;
}
```

Withdrawal:

```
if (amount <= 0) { ... }
else if (amount > balance) { ... }
else { balance -= amount; }
```